Unidad 2

Actividad 1

En el programa del puente, en el puente solo se puede cruzar en un sentido. A partir de él, crea un programa que controle el paso del puente en ambos sentidos. Como máximo, puede haber cuatro personas en el puente y tres en cada sentido. Puedes no multiplicar los tiempos de espera por 1000 para que la simulación avance más rápidamente.

PasoPorPuente.java

```
import java.util.Random;
public class PasoPorPuente {
    public static void main(String[] args) {
        final Puente puente = new Puente();
        int tMinParaLlegadaPersona = 1;
        int tMaxParaLlegadaPersona = 30;
        int tMinPaso = 10;
        int tMaxPaso = 50;
        int minPesoPersona = 40;
        int maxPesoPersona = 120;
        System.out.println(">>>>>>>>>>>>> Comienza simulación.");
        Random r = new Random();
        int idPersona = 1;
        while (true) {
            int tParaLlegadaPersona = tMinParaLlegadaPersona + r.nextInt(tMaxParaLlegadaPersona -
tMinParaLlegadaPersona + 1);
            int pesoPersona = minPesoPersona + r.nextInt(maxPesoPersona - minPesoPersona + 1);
            int dir =  r.nextInt(0,1+1);
            String Dir = (dir == 0) ? "izquierda" : "derecha";
            System.out.printf("Siguiente persona llega en %d segundos por
la %s.\n",tParaLlegadaPersona,Dir);
            try {
                Thread.sleep(250 * tParaLlegadaPersona);
            } catch (InterruptedException ex) {
                System.out.printf(
                        "Interrumpido proceso principal");
            }
            Thread hiloPersona =
                    new Thread(
                            new Persona(
                                    puente,
                                    pesoPersona,
                                    tMinPaso,
                                    tMaxPaso,
                                    "P" + idPersona,
                                    dir));

            hiloPersona.start();

            idPersona++;
        }
    }
}
```

Persona.java

```
import java.util.Random;
class Persona implements Runnable {
    private final String idPersona;
    private final int peso;
    private final int tMinPaso;
    private final int tMaxPaso;
    private final Puente puente;
    private final int dir;
    public int getPeso() {
        return peso;
    }
    public int getDir() {
        return dir;
    }
    Persona(
            Puente puente,
            int peso,
            int tMinPaso,
            int tMaxPaso,
            String idP,
            int dir) {
        this.puente = puente;
        this.peso = peso;
        this.tMinPaso = tMinPaso;
        this.tMaxPaso = tMaxPaso;
        this.idPersona = idP;
        this.dir = dir;
    }
    @Override    public void run() {
        String Dir = (dir != 0) ? "izquierda" : "derecha";
        System.out.printf(
                "-- %s de %d kg quiere cruzar hacia la %s, en puente %d kg, %d personas.\n",
                this.idPersona,
                this.peso,
                Dir,
                puente.getPeso(),
                puente.getNumPersonas());
        boolean autorizado = false;
        while (!autorizado) {
            synchronized (this.puente) {
                autorizado =
                        this.puente.autorizacionPaso(this);
                if (!autorizado) {
                    try {
                        System.out.printf(
                                "# %s debe esperar.\n",
                                this.idPersona);
                        this.puente.wait();
                    } catch (InterruptedException ex) {
                        System.out.printf(
                                "Interrupción mientras %s espera.\n",
                                this.idPersona);
}}}}
        System.out.printf(
                "-> %s con peso %d puede cruzar, peso en puente %d con %d personas.\n",
                this.idPersona,
                this.getPeso(),
                this.puente.getPeso(),
                this.puente.getNumPersonas());
        Random r = new Random();
        int tiempoPaso =
                this.tMinPaso                    + r.nextInt(this.tMaxPaso - this.tMinPaso + 1);
        try {System.out.printf("%s va a tardar tiempo %d en cruzar.\n",this.idPersona,tiempoPaso);
            Thread.sleep(250 * tiempoPaso);
        } catch (InterruptedException ex) {
            System.out.printf(
                    "Interrupción mientras %s pasa.\n",
                    this.idPersona);}
        synchronized (this.puente) {
            this.puente.terminaPaso(this);
            System.out.printf("<- %s sale del puente, puente soporta peso %d, %d personas.\n",this.idPersona,this.puente.getPeso(),
this.puente.getNumPersonas());
            puente.notifyAll();}}}
```

Puente.java

```
class Puente {

    private static final int PESO_MAXIMO = 200;
    private static final int MAX_PERSONAS = 4;
    private static final int MAX_PERSONAS_DIR = 3;

    private int peso = 0;
    private int numPersonas = 0;
    private int numPersonas0 = 0;
    private int numPersonas1 = 0;

    synchronized public int getPeso() {
        return peso;
    }

    synchronized public int getNumPersonas() {
        return numPersonas;
    }

    synchronized public boolean autorizacionPaso(Persona persona) {

        boolean result;
        int personasDir = 0;
        if (persona.getDir() == 0){
            personasDir = numPersonas0;
        } else {
            personasDir = numPersonas1;
        }
        if (this.peso + persona.getPeso() <= PESO_MAXIMO              && this.numPersonas <
MAX_PERSONAS && personasDir < MAX_PERSONAS_DIR) {

            this.numPersonas++;
            if (persona.getDir() == 0){
                numPersonas0++;
            } else {
                numPersonas1++;
            }
            this.peso += persona.getPeso();
            result = true;

        } else {
            result = false;
        }

        return result;
    }

    synchronized public void terminaPaso(Persona persona) {
        this.peso -= persona.getPeso();
        this.numPersonas--;
        if (persona.getDir() == 0){
            numPersonas0--;
        } else {
            numPersonas1--;
        }
    }
}
```